ELEMENTOS BÁSICOS DE LA ORIENTACIÓN A OBJETOS

OBJETOS Y CLASES

Un programa orientado a objetos se compone de objetos que se comunican entre sí intercambiando mensajes.

¿Dónde están las clases?

Aunque prácticamente todos los lenguajes orientados a objetos tienen clases en realidad no son necesarias.

La Programación Basada en Prototipos es un estilo de programación orientada a objetos en el cual, las clases no están presentes y la herencia se consigue clonando objetos ya existentes (que sirven de prototipos) y extendiendo sus funcionalidades.

Definición de una clase:

Una clase es una plantilla que describe la estructura y el comportamiento de un tipo de objetos y permite la creación de los mismos.

La definición de una clase abarca:

- **Estructura**: Define el estado (atributos) y el comportamiento (métodos) de los objetos de la clase.
- **Relaciones**: Define las relaciones (herencia) de esa clase con otras clases existentes.
- **Propósito de creación de nuevos objetos**: Define los procedimientos para construir nuevos objetos de esa clase (métodos constructores).

Definición de objeto:

Un objeto es un elemento identificable que contiene características declarativas (que determinan su estado) y características procedimentales (que modelan su comportamiento).

ESTADO DE OBJETOS

Ortodoxia de la orientación a objetos:

Los atributos deben declararse privados y su acceso sólo debe ser posible a través de métodos públicos de lectura/escritura.

Motivación de los atributos privados:

- Control de acceso.
- Abstracción de la implementación.
- Limitar la propagación de modificaciones.

Atributos estáticos

Atributos que pertenecen a la clase y no a una instancia. Por lo tanto, son compartidos por todas las instancias de la clase. Cuando cambie, cambiará para todas las instancias. Se acceden anteponiendo el nombre de la clase al nombre del atributo.

Atributos finales

Atributos constantes: una vez se les ha asignado un valor no es posible cambiarlo.

@informaticapau

Constantes de clase

Las constantes suelen ser atributos de clase ya que al no poder modificarse su valor no tiene sentido almacenarlo en cada instancia. Permiten desarrollar el código de forma más clara. Por estilo se representan en mayúsculas.

Constantes de instancia (*blank finals*)

Las constantes se declaran, pero no se les da un valor en la definición, sino que este valor se asigna en el constructor. Permite que tengamos constantes con valores diferentes para cada instancia.

COMPORTAMIENTO DE OBJETOS

Mensajes y métodos

Si un objeto acepta un mensaje entonces acepta la responsabilidad de llevar a cabo la acción que indica ese mensaje mediante la ejecución de un método.

Paso de parámetros

Definiciones de parámetros

- **Parámetros formales**: Son los que se declaran en la implementación del método.
- **Parámetros actuales**: Son las variables que se incluyen en la llamada o invocación al método.

Tipo de paso parámetros

- **Paso por valor**: El valor de los parámetros actuales se copia en los parámetros formales (los cambios en el método no se propagan al exterior).
- **Paso por referencia**: El parámetro formal apunta a la misma dirección de memoria que el parámetro actual (los cambios en el método se propagan al exterior).

Ensombrecimiento

Los parámetros ensombrecen el nombre de los atributos en caso de tener el mismo nombre. Esto quiere decir que, dentro de un método, en caso de ambigüedad, se entenderá que nos estamos refiriendo al elemento más cercano (el parámetro).

Métodos de clase

Métodos que no se ejecutan sobre una instancia en particular de una clase sino sobre la clase misma. Pueden ser ejecutados, aunque no haya ninguna instancia creada. Se llaman anteponiendo al método el nombre de la clase.

Métodos constructores

Son los métodos que se utilizan para crear e inicializar instancias.

Encadenamiento de constructores

Un método puede tener varios constructores siempre que su número o tipo de parámetros sea distinto.

Métodos destructores y recolección de basura:

- **Métodos destructores**: Los métodos destructores sirven para liberar la memoria que ocupan los objetos en el montículo (heap) y dejarla disponible para ser reservada para otros objetos.
- **Recolección de basura**: Proceso automático y asíncrono que detecta objetos “basura” en el montículo para liberar su memoria reservada. Un objeto es basura cuando está desreferenciado, es decir, ha perdido la referencia de la pila que apuntaba él por lo que no

@informaticapau

es accesible desde el programa que lo usaba. La recolección de basura libera al programador de la responsabilidad del manejo de la memoria.

IDENTIDAD DE OBJETOS

La identidad es la propiedad de un objeto que lo distingue de todos los demás de forma independiente al valor de sus atributos.

- Se consigue a través de identificadores de objetos (OID – object identifier)
- Son independientes de los atributos que definen el estado de los objetos.
- Son generados por el sistema no pueden ser modificados por el usuario.

Tipos de comparación de objetos:

- **Identidad**: Dos objetos son idénticos si y solo si son el mismo objeto (es decir, tienen un mismo OID).
- **Igualdad**: Dos objetos se consideran iguales si, a pesar de ser objetos distintos (con distinto OID), sus atributos son idénticos.

Igualdad en objetos compuestos

- Igualdad superficial (**shallow**): Dos objetos son superficialmente iguales si sus atributos son idénticos.
- Igualdad profunda (**deep**): Dos objetos son profundamente iguales si sus atributos son recursivamente iguales.

Funciones hash

Son funciones que mapean datos de un tamaño arbitrario en datos de un tamaño fijo.

Copia de objetos compuestos

- **Copias superficiales**: Son fáciles de hacer, pero son peligrosas (estamos compartiendo referencias a los objetos internos).
- **Copias profundas**: Son más seguras. Consumen más tiempo y memoria dependiendo del grado de composición de los objetos. Una buena idea suele ser definir constructores de copia en los que creamos objetos iguales a otros objetos que nos pasan por parámetro.

Acceso a objetos privados mutables

Si devolvemos referencias a objetos privados mutables, el contenido de dichos objetos puede ser modificado desde fuera de la clase en la que residen.

Posibles soluciones a este problema son no devolver directamente las referencias (usar clones) o hacer objetos que no puedan ser modificados una vez creados (inmutables). Los objetos inmutables son simples, pueden compartirse libremente y son *thread-safe* (múltiples hilos pueden acceder a ellos concurrentemente sin necesitar sincronización).

TIPO ENUMERADO

Es un tipo de datos que consiste en un conjunto de valores con nombre llamados elementos que se comportan como constantes en el lenguaje.

@informaticapau